

## 0 Conventions & Math Toolkit

- **Height = number of edges** on the longest root-to-leaf path. Single node:  $h = 0$ . Empty:  $h = -1$ .
- Heap arrays are **0-indexed**. BST insert sends **equal keys left**. BST delete uses the **successor**.
- B-tree **order**  $m$  (Knuth) = max children. PSO 5 uses **minimum degree**  $t$  (CLRS).

### Sums you need

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \in \Theta(n^2) \quad \sum_{i=0}^h 2^i = 2^{h+1} - 1$$
$$\sum_{i=0}^{\infty} \frac{1}{2^i} = 2 \quad \sum_{i=1}^{\infty} \frac{i}{2^i} = 2 \quad (\text{a constant!})$$
$$\sum_{i=1}^n \log i = \log(n!) \in \Theta(n \log n) \quad \log_b n = \frac{\log_2 n}{\log_2 b}$$

**Trap:** a geometric (or  $\frac{i}{2^i}$ -weighted) sum **converges to a constant**. It never produces a  $\log n$  factor.

### Asymptotic bounds

$$O = \text{upper bound } (\leq) \quad \Omega = \text{lower bound } (\geq) \quad \Theta = \text{both}$$
$$n \log n \in O(n^2) \text{ is TRUE} \quad \Theta(n \log n) \neq \Theta(n^2)$$

### Necessarily true / Possibly true / Necessarily false

1. Does the claim depend on the **input** or a **parameter** ( $k$ ,  $m$ , order, shape)? Find one YES case and one NO case  $\Rightarrow$  **Possibly true**.
  2. It can **never fail** (definition, theorem, or  $O(\text{something bigger})$ )  $\Rightarrow$  **Necessarily true**.
  3. It can **never hold** (contradicts a theorem or definition)  $\Rightarrow$  **Necessarily false**.
- Examples:** HeapSort is  $O(n^2)$ : **N.T.** HeapSort is  $\Theta(n^2)$ : **N.F.** Counting sort is  $O(n)$ : **P.T.** BST search is  $O(\log n)$ : **P.T.** B-tree height is  $O(\log n)$ : **N.T.** Comparison sort in  $O(n)$  worst case: **N.F.**
- How to justify: False:** "Counterexample: input ... produces ..."
- Necessarily true:** "By the definition / theorem ..., this holds for every input."
- Possibly true:** "True when ... (example), false when ... (example)."
- Runtime:** "Loop 1 is  $O(\cdot)$ ; loop 2 runs  $\sum \dots$  times in total; so  $O(\cdot)$ ."

### How to: analyze a brand-new algorithm (Part A T/F)

1. Cost each loop separately.
2. **Nested-loop trap:** add up the inner loop's *total* iterations over all outer iterations. If the inner loop runs  $C[v]$  times, then  $\sum_v C[v] = n$ , so the total is  $O(n + k)$ , **not**  $O(nk)$ .
3. Any extra array or list of size  $n$  or  $k \Rightarrow$  **not in place**.
4. Stable? Follow two equal keys. Does it move records, or just rewrite values from counts (losing payloads)?
5. "Best scenario" = small key range  $k$  and large  $n$ .

## 1 Binary Heap

**Definition.** A **complete** binary tree (every level full except the last, which fills left  $\rightarrow$  right) with **heap order**:

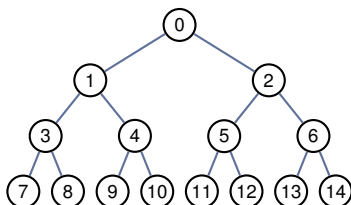
- **Min-heap:** parent  $\leq$  each child. **Max-heap:** parent  $\geq$  each child.
- Siblings are **unordered**. A heap is **not** a BST, and its array is **not** sorted.
- Priority-queue ADT: Insert, Extract/Delete Min/Max, Peek, IsEmpty, Size.

### Array formulas (root at index 0)

#### Index math

$$\text{left}(i) = 2i + 1 \quad \text{right}(i) = 2i + 2$$
$$\text{parent}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor \quad \text{last internal} = \left\lfloor \frac{n}{2} \right\rfloor - 1$$
$$\text{leaves} : \left\lfloor \frac{n}{2} \right\rfloor \text{ to } n-1 \quad \# \text{leaves} = \left\lceil \frac{n}{2} \right\rceil$$
$$\text{height } h = \lfloor \log_2 n \rfloor \quad 2^h \leq n \leq 2^{h+1} - 1$$

**Example:**  $i = 23$ : left = 47, right = 48, parent = 11.  $n = 99$ : last internal = 48, and  $64 \leq 99 \leq 127 \Rightarrow h = 6$ .



Array index picture. Row  $d$  holds indices  $2^d - 1$  to  $2^{d+1} - 2$ .

### Code (min-heap; for a max-heap flip every comparison)

```
SiftUp(A, i):
  while i > 0 and A[parent(i)] > A[i]: // child smaller -> swap
    swap(A, i, parent(i)); i <- parent(i)

Insert(x):
  if n = capacity: A <- Resize(A, 2*capacity) // amortized O(1)
  A[n] <- x; n <- n + 1; SiftUp(A, n - 1)

MinChildIndex(A, n, i):
  lc <- 2i + 1; if lc >= n: return n // no children
  rc <- 2i + 2; if rc >= n: return lc // only a left child
  if A[lc] < A[rc]: return lc else: return rc // TIE -> RIGHT

SiftDown(A, n, i):
  m <- MinChildIndex(A, n, i)
  while m < n and A[i] > A[m]:
    swap(A, i, m); i <- m; m <- MinChildIndex(A, n, i)

ExtractMin():
  if n = 0: throw EmptyHeap
  t <- A[0]; A[0] <- A[n-1]
  SiftDown(A, n - 1, 0); n <- n - 1; return t
```

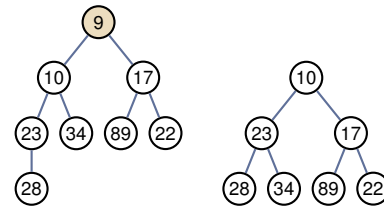
### Trap: slide typo

The slide prints  $A[p] < A[i]$  in the *min-heap* SiftUp. That is the **max-heap** comparison. Reason from the rule, not the symbol.

### Costs

| Operation                              | Cost                             |
|----------------------------------------|----------------------------------|
| Peek (min in min-heap)                 | $O(1)$                           |
| Insert, Extract                        | $O(\log n)$ (one root-leaf path) |
| Build with $n$ inserts (top-down)      | $O(n \log n)$                    |
| Build with Heapify (bottom-up)         | $O(n)$                           |
| Search any key; find max in a min-heap | $O(n)$                           |

### Worked example: min-heap inserts 23, 10, 17, 28, 34, 89, 22, 9



Left: after all inserts, array [9,10,17,23,34,89,22,28]. Right: after ExtractMin (returns 9): 28 moves to the root, swaps with 10, then 23.

### Facts that become trick questions

- The **max of a min-heap is a leaf** (could be any leaf).
- Distinct keys: the 2nd smallest is a **child of the root**; the 3rd smallest is at depth 1 or 2; the  $k$ -th smallest is at depth  $\leq k - 1$ .
- A sorted ascending array **is** a min-heap. A min-heap array is only *possibly* sorted.
- **Number of places a new item can end up**  $= h + 1$ , where  $h$  is the height *after* inserting. ( $n = 99 \rightarrow 100$  nodes,  $h = 6$ , answer 7.)

### How to: heap trace questions

**Insert:** put at index  $n$ ; compare with parent  $\lfloor \frac{i-1}{2} \rfloor$ ; swap while order is violated. Write the array after *every* insert.

**Extract:** save root; move last item to root; shrink; swap with the **smaller** child (min) or **larger** child (max) until it fits.

**Is it a heap?** For each  $i \leq \lfloor n/2 \rfloor - 1$  compare  $A[i]$  with  $A[2i + 1]$  and  $A[2i + 2]$ . Example: [10, 20, 15, 30, 40, 50, 12]:  $A[2] = 15 > A[6] = 12$ , so **not** a heap.

**Traversal of a heap:** draw the tree level by level from the array, then traverse. PSO 3: max-heap inserts 8, 11, 3, 10, 7, 4, 12  $\rightarrow$  [12, 10, 11, 8, 7, 3, 4], in-order 8, 10, 7, 12, 3, 11, 4.

## 2 Heapify (build a heap in place)

```
Heapify(A):
  n <- size of A
  for i from floor(n/2) - 1 down to 0:
    SiftDown(A, n, i) // leaves are already heaps
```

- Bottom-up, in place,  $\Theta(n)$ .
- Number of SiftDown calls  $= \lfloor n/2 \rfloor$ . First index  $= \lfloor n/2 \rfloor - 1$  (e.g.  $n = 13 \rightarrow 5$ ).
- Max swaps in one SiftDown from the root  $= h$ .

Why Heapify is  $O(n)$

At most  $\lceil n/2^{h+1} \rceil$  nodes have height  $h$ , and each does  $O(h)$  work:

$$\sum_{h=0}^{\lceil \log_2 n \rceil} \frac{n}{2^{h+1}} \cdot h = \frac{n}{2} \sum_{h \geq 0} \frac{h}{2^h} = \frac{n}{2} \cdot 2 = n \in O(n)$$

Worked example: max-heapify [4, 6, 3, 5, 7, 1] ( $n = 6, i = 2, 1, 0$ )

| <i>i</i> | Array after        | What happened                                                     |
|----------|--------------------|-------------------------------------------------------------------|
| 2        | [4, 6, 3, 5, 7, 1] | 3 vs child 1: no swap                                             |
| 1        | [4, 7, 3, 5, 6, 1] | 6 vs 5, 7: swap with 7                                            |
| 0        | [7, 6, 3, 5, 4, 1] | 4 $\leftrightarrow$ 7, then 4 $\leftrightarrow$ 6 (keep sinking!) |

Trap: Heapify  $\neq$  inserting one by one

Inserting 4, 6, 3, 5, 7, 1 into a max-heap gives [7, 6, 3, 4, 5, 1], but Heapify gives [7, 6, 3, 5, 4, 1]. Both are valid. "Same result" is only **possibly** true.

3 HeapSort

HeapSort(A):

n <- size of A  
Heapify(A) // MAX-heap for ascending order  
for i from n-1 down to 1:  
  swap(A, 0, i) // current max -> its final slot  
  n <- n - 1 // shrink heap: protects sorted part  
  SiftDown(A, n, 0) // MaxChildIndex; swap if A[i] < A[m]

- Ascending  $\Rightarrow$  **max**-heap. Descending  $\Rightarrow$  min-heap.
- The array is always **[ heap | sorted suffix ]**.

$$T(n) = \underbrace{O(n)}_{\text{Heapify}} + \underbrace{(n-1) \cdot O(\log n)}_{\text{sort-down}} = O(n \log n)$$

|                      |                                          |
|----------------------|------------------------------------------|
| Worst / average time | $O(n \log n)$                            |
| All keys equal       | $\Theta(n)$ (SiftDown stops immediately) |
| Extra space          | $\Theta(1)$ (in place)                   |
| Stable               | <b>No</b>                                |
| In practice          | poor cache behaviour                     |

Trace: sort-down from the heap [7, 6, 3, 5, 4, 1]

| <i>i</i> | After swap          | After SiftDown           |
|----------|---------------------|--------------------------|
| 5        | [1, 6, 3, 5, 4   7] | [6, 5, 3, 1, 4   7]      |
| 4        | [4, 5, 3, 1   6, 7] | [5, 4, 3, 1   6, 7]      |
| 3        | [1, 4, 3   5, 6, 7] | [4, 1, 3   5, 6, 7]      |
| 2        | [3, 1   4, 5, 6, 7] | [3, 1   4, 5, 6, 7]      |
| 1        | [1   3, 4, 5, 6, 7] | done: [1, 3, 4, 5, 6, 7] |

Not stable: counterexample

Input [2<sub>a</sub>, 2<sub>b</sub>, 1]. Heapify: 2<sub>a</sub> is not smaller than 2<sub>b</sub>, so no swap.  
*i* = 2: swap  $\rightarrow$  [1, 2<sub>b</sub>, 2<sub>a</sub>], sift  $\rightarrow$  [2<sub>b</sub>, 1, 2<sub>a</sub>]. *i* = 1: swap  $\rightarrow$  [1, 2<sub>b</sub>, 2<sub>a</sub>].  
Output has 2<sub>b</sub> before 2<sub>a</sub>: the order of equal keys changed.

Variants from the PSOs / tests

- PSO 4 bags:**  $n/k$  bags of  $k$ . HeapSort each ( $O(n \log k)$ ), merge pairwise ( $O(n \log \frac{n}{k})$ ): total  $O(n \log n)$ .
- MinHeapSort** (insert all into heap  $H$ , extract into  $B$ ):  $O(n \log n)$  time,  $\Theta(n)$  extra space, not stable. Using Heapify instead of inserts does **not** improve the worst case, because the  $n$  extractions are still  $\Theta(n \log n)$ .
- $k$ -sorted array** (each item at most  $k$  from its final slot): keep a min-heap of  $k+1$  items  $\Rightarrow O(n \log k)$  time,  $\Theta(k)$  space. No contradiction with the lower bound: only  $\leq (2k+1)^n$  inputs are possible.

4 Sorting Fundamentals

**Four things to analyze:** correctness, running time (# compares and swaps), **stability**, space (**in place** =  $O(1)$  extra).

**Stable:** equal keys keep their input order. Stability is what makes radix sort work.

**Decision-tree lower bound**

A tree of height  $h$  has at most  $2^h$  leaves, and it needs at least  $n!$  leaves:

$$\begin{aligned} 2^h \geq n! &\Rightarrow h \geq \log_2(n!) = \sum_{i=1}^n \log_2 i \\ &\geq \sum_{i=n/2}^n \log_2 i \geq \frac{n}{2} \log_2 \frac{n}{2} \in \Omega(n \log n) \end{aligned}$$

Theorem

Every **comparison** sort needs  $\Omega(n \log n)$  comparisons in the **worst case**. HeapSort and MergeSort are optimal.  
Smallest possible worst case =  $\lceil \log_2 n! \rceil$ :  $n = 3: 3$   $n = 4: 5$   $n = 5: 7$   $n = 6: 10$ .

**Ways around it:** non-comparison sorts (counting, bucket, pigeonhole, radix) use keys as indices. Restricted inputs (e.g.  $k$ -sorted) have fewer possible permutations.

Master comparison table

| Sort        | Time                                            | Space         | Stable      | Compares   |
|-------------|-------------------------------------------------|---------------|-------------|------------|
| HeapSort    | $O(n \log n)$                                   | $\Theta(1)$   | No          | Yes        |
| MergeSort   | $\Theta(n \log n)$                              | $\Theta(n)$   | Yes         | Yes        |
| QuickSort   | avg $n \log n$ , worst $n^2$                    | $O(\log n)$   | No          | Yes        |
| Insertion   | $O(n^2)$ , best $O(n)$                          | $\Theta(1)$   | Yes         | Yes        |
| Counting    | $\Theta(n+k)$                                   | $\Theta(n+k)$ | Yes         | No         |
| Pigeonhole  | $O(n+m)$                                        | $O(n+m)$      | Yes         | No         |
| Bucket      | avg $O(n + \frac{n^2}{k} + k)$ , worst $O(n^2)$ | $O(n+k)$      | if inner is | inner sort |
| Radix (LSD) | $O(d(n+b))$                                     | $O(n+b)$      | Yes         | No         |

How to: pick the right sort

- Small integer range ( $k \in O(n)$ ), needs stability  $\rightarrow$  **counting sort**.
- Huge range, few items (e.g. IDs up to  $10^9$ )  $\rightarrow$  **merge / heap / quick**.
- Uniform reals in  $[0, 1]$   $\rightarrow$  **bucket sort**. Fixed number of digits  $\rightarrow$  **radix**.
- $O(1)$  extra space + guaranteed  $n \log n \rightarrow$  **heap**. Stable comparison sort  $\rightarrow$  **merge**.

5 Counting Sort

**Requirement:** keys are integers in  $[0, k]$ .

CountingSort(A, k):

C <- array of k+1 zeros // O(k)  
for i from 0 to n-1: C[A[i]]++ // O(n) C[v] = # equal to v  
for i from 1 to k: C[i] += C[i-1] // O(k) C[v] = # <= v  
B <- array of size n  
for i from n-1 DOWNTO 0: // O(n) BACKWARDS = stable  
  B[C[A[i]] - 1] <- A[i]  
  C[A[i]] <- C[A[i]] - 1  
return B // optional: copy B into A

Time  $\Theta(n+k)$  Space  $\Theta(n+k)$  Stable Not in place  
**It is  $O(n)$  only when  $k \in O(n)$ .** After prefix sums,  $C[v]$  is one past the last slot for  $v$ . After placement,  $C[v]$  is the first slot of  $v$ 's block.

Lecture trace ( $k = 5$ )

$A = (2, A) (5, C) (3, H) (0, B) (2, X) (3, B) (0, X) (3, D)$

| <i>v</i>    | 0 | 1 | 2 | 3 | 4 | 5 |
|-------------|---|---|---|---|---|---|
| counts      | 2 | 0 | 2 | 3 | 0 | 1 |
| prefix sums | 2 | 2 | 4 | 7 | 7 | 8 |

| <i>i</i>      | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
|---------------|----|----|----|----|----|----|----|----|
| A[i]          | 3D | 0X | 3B | 2X | 0B | 3H | 5C | 2A |
| C[key] before | 7  | 2  | 6  | 4  | 1  | 5  | 8  | 3  |
| slot = C - 1  | 6  | 1  | 5  | 3  | 0  | 4  | 7  | 2  |

**Result:**  $B = (0, B) (0, X) (2, A) (2, X) (3, H) (3, B) (3, D) (5, C)$ .

Trap: forward last loop breaks stability

$A = [(3, a), (1, b), (3, c)]$ , prefix  $C = [0, 1, 1, 3]$ .  
Forward:  $(3, a) \rightarrow B[2], (1, b) \rightarrow B[0], (3, c) \rightarrow B[1] \Rightarrow (1, b), (3, c), (3, a)$ : **un-stable**.  
Backward:  $(1, b), (3, a), (3, c)$ : **stable**.

Offsets, range queries, bad cases

**Negative / shifted keys:**

$\text{index}(v) = v - \min$  array size =  $\max - \min + 1$   
[−100, 20]: index =  $v + 100$ , size 121. [−50, 30]: size 81, and  $-7 \mapsto 43$ .

**Range query** in  $O(1)$  after  $\Theta(n+k)$  prep (prefix sums):

$$\begin{aligned} \#\{a \leq x \leq b\} &= C[b] - C[a-1] \quad (\text{or } C[b] \text{ if } a = 0) \\ \#\{x \geq a\} &= n - C[a-1] \end{aligned}$$

Trap: check your  $k$

- 1,000 IDs in  $[0, 10^9]$ : a  $10^9$ -entry array ( $\approx 4$  GB) and  $10^9$  steps to zero it, vs.  $n \log n \approx 10^4$ . Use merge / heap / quick.
- $k$ -digit numbers with  $k \in \Theta(n)$ : max value  $10^k - 1$ , so time  $O(n + 10^n)$ , **exponential**.
- Keys in  $[0, n^2]$ : time  $\Theta(n^2)$ , never  $O(n)$ .

**TallySort** (rebuild  $A$  from counts):  $O(n + k)$ , **not**  $O(nk)$ ; loses payloads, so it cannot sort records.

## 6 Bucket, Pigeonhole & Radix Sort

```
BucketSort(A, k):           // k buckets
  B <- array of k empty lists
  M <- max(A) + 1
  for i from 0 to n-1:
    j <- floor(k * A[i] / M) // bucket index 0..k-1
    B[j].insert(A[i])
  for i from 0 to k-1: sort list B[i]
  return B[0] ++ B[1] ++ ... ++ B[k-1]
```

$$j = \left\lfloor \frac{k \cdot A[i]}{M} \right\rfloor, \quad M = \max(A) + 1$$

Classic version: keys in  $[0, 1)$ ,  $k = n$ ,  $j = \lfloor n \cdot A[i] \rfloor$ . Correct because  $j$  never decreases as the key grows.

**Trace:** [29, 25, 3, 49, 9, 37, 21, 43],  $k = 8$ ,  $M = 50$

|         |      |     |      |      |      |      |      |      |
|---------|------|-----|------|------|------|------|------|------|
| key     | 29   | 25  | 3    | 49   | 9    | 37   | 21   | 43   |
| $8x/50$ | 4.64 | 4.0 | 0.48 | 7.84 | 1.44 | 5.92 | 3.36 | 6.88 |
| bucket  | 4    | 4   | 0    | 7    | 1    | 5    | 3    | 6    |

Bucket 4 = [29, 25]  $\rightarrow$  [25, 29]. Result [3, 9, 21, 25, 29, 37, 43, 49].

Running time

Best:  $O(n)$  (about one item per bucket)

Average (uniform):  $O\left(n + \frac{n^2}{k} + k\right) \xrightarrow{k=n} O(n)$

Worst:  $O(n^2)$  (all in one bucket + insertion sort)

One key per bucket:  $O(n + k)$

With heap/merge sort inside the buckets, the worst case drops to  $O(n \log n)$ .

**Stable** if and only if the inner sort is stable and inserts append. Counting sort = one bucket per value.

How to: bucket-index questions

Compute  $\lfloor kx/M \rfloor$  exactly. Check with multiplication:  $5 \cdot 55 = 275 < 276 = 3 \cdot 92$ , so  $\lfloor 275/92 \rfloor = 2$ . Insert in **input order**; sort each bucket; concatenate. The max always lands in bucket  $k - 1$ , never  $k$ .

**Words by length in  $O(n)$ :** 45 lists (longest word), a word of length  $l$  goes in list  $l - 1$ , join the 45 lists (a constant).

### Pigeonhole sort (Problem Set 2)

- $m = \max - \min + 1$  holes; append  $A[i]$  to hole  $H[A[i] - \min]$ ; read holes in order.
- Time and space  $O(n + m)$ , stable, not in place.
  - Never**  $O(n^2)$ : no sorting inside holes; all  $n$  in one hole is fine  $\Rightarrow$  “degrades to  $n^2$ ” is N.F.
  - Can waste memory:** [10, 1, 1, ..., 1]. Best: huge  $n$ , tiny range (500k ints in 1..100). Bad: few items with a huge range; floats.

### Radix sort (LSD)

Stable counting sort on each digit, least significant first.  $d$  digits in base  $b$ :  
 $T = O(d(n + b))$

**PSO 5** ( $m$  integers, 1 to  $k$  digits,  $k \in \Theta(m)$ ,  $n$  total digits, uniform lengths): bucket by digit count ( $B[d_i - 1]$ ), radix-sort each bucket, concatenate (fewer digits  $\Rightarrow$  smaller).

$$\sum_{i=0}^{k-1} (i + 1) \cdot \frac{m}{k} = O(n) \quad \text{since } \frac{m}{k} \text{ is a constant}$$

## 7 Binary Search Tree

**Definition:** null, or a root with left and right BSTs where **every** key in the left subtree  $\leq$  root  $<$  **every** key in the right subtree. **In-order traversal = sorted.**

```
search(r, x):
  if r = null: return null
  if x = r.item: return r
  if x < r.item: return search(r.left, x)
  return search(r.right, x)

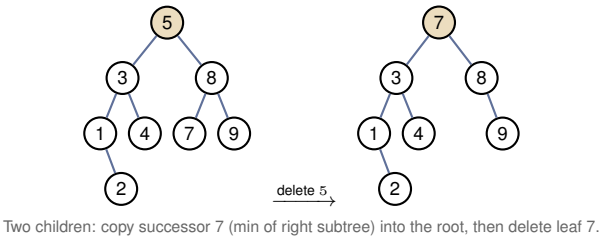
insert(r, x):
  if r = null: return new Node(x)
  if x <= r.item: r.left <- insert(r.left, x) // equal -> LEFT
  else: r.right <- insert(r.right, x)
  return r

delete(r, x):
  if r = null: return null
  if x < r.item: r.left <- delete(r.left, x)
  elif x > r.item: r.right <- delete(r.right, x)
  else:
    if r.left = null: return r.right // 0 or 1 child
    if r.right = null: return r.left
    s <- findmin(r.right) // in-order successor
    r.item <- s.item
    delete(r.right, s.item)
    return r
```

```
r.item <- s.item
r.right <- delete(r.right, s.item)
return r

findmin(r): while r.left != null: r <- r.left
return r

generateBST(A, l, r): // A is sorted
  if r < l: return null
  m <- floor((l + r) / 2)
  root <- new Node(A[m])
  root.left <- generateBST(A, l, m - 1)
  root.right <- generateBST(A, m + 1, r)
  return root
```



### Running time and height

|                              | Unbalanced (worst) | Balanced      |
|------------------------------|--------------------|---------------|
| search / insert / delete one | $O(n)$             | $O(\log n)$   |
| insert all $n$               | $O(n^2)$           | $O(n \log n)$ |

Balanced:  $|h(\text{left}) - h(\text{right})| \leq 1$  at every node  $\Rightarrow h \in O(\log n)$

$$h + 1 \leq n \leq 2^{h+1} - 1 \Rightarrow \lfloor \log_2 n \rfloor \leq h \leq n - 1$$

Sorted (or reverse-sorted) inserts build a list,  $h = n - 1$ :

$$\text{total cost} = \sum_{i=1}^n (i - 1) = \frac{n(n - 1)}{2} \in \Theta(n^2)$$

### Facts

- The successor of a node with two children (min of right subtree) has **no left child**.
- The max node has no right child. Shape depends on insertion order.
- Inserting a BST's pre-order sequence rebuilds the same tree.
- Heap vs BST vs B-tree: heap search  $O(n)$ , BST  $O(h)$ , B-tree  $O(\log n)$ . Heaps and BSTs grow at the bottom; B-trees grow at the **top**.

Trap: “search-path” claim is FALSE (PSO 5)

Search for 4. Left of the path  $A = \{2\}$ , on the path  $B = \{1, 3, 4\}$ , right  $C = \emptyset$ . The claim  $a \leq b \leq c$  fails because  $2 > 1$ .

How to: BST problem types

**Build + height:** insert each key from the root ( $\leq$  goes left); height = longest path in **edges**. PS 2: 4, 2, 8, 0, 1, 3  $\rightarrow h = 3$  (4-2-0-1). A balanced BST on 6 keys has  $h = \lfloor \log_2 6 \rfloor = 2$ .

**Delete (two children):** successor = go right once, then left until null. Copy it up, delete it from the right subtree (it has at most one child).

**Traversals:** pre = N L R, in = L N R, post = L R N, level-order = BFS.

**generateBST:** root =  $A[\lfloor (l + r)/2 \rfloor]$ , recurse on halves; height  $\lfloor \log_2 n \rfloor$ .

## 8 B-Tree

Bayer & McCreight (1972), for data on disk: 1 node = 1 disk block, so nodes are wide and the tree is shallow. **Always perfectly balanced**; grows at the **root**.

### Rules for order $m$

|                                   | Order $m$                                             | 2-3 ( $m = 3$ ) | 2-3-4 ( $m = 4$ ) |
|-----------------------------------|-------------------------------------------------------|-----------------|-------------------|
| max children                      | $m$                                                   | 3               | 4                 |
| max keys                          | $m - 1$                                               | 2               | 3                 |
| min children (internal, non-root) | $\lceil m/2 \rceil$                                   | 2               | 2                 |
| min keys (non-root)               | $\lceil m/2 \rceil - 1$                               | 1               | 1                 |
| root                              | $\geq 2$ children unless it is a leaf ( $\geq 1$ key) |                 |                   |

- A node with  $c$  children has exactly  $c - 1$  keys, kept sorted.
- Key  $i$  separates child  $i$  (smaller keys) from child  $i + 1$  (larger keys).
- All leaves are at the same depth**; internal nodes have no null children.

Minimum degree t (PSO 5) and key counts

Non-root nodes hold  $t - 1$  to  $2t - 1$  keys ( $t$  to  $2t$  children); the root holds  $\geq 1$  key.  
 $t = 2$  is the 2-3-4 tree.

Keys in a B-tree of height h

$\max \text{ keys} = (2t)^{h+1} - 1$

$\min \text{ keys} = 2t^h - 1$

order m,  $\max = m^{h+1} - 1$

2-3 tree :  $2^{h+1} - 1$  to  $3^{h+1} - 1$

Check ( $t = 2$ ):  $h = 1$ : 3 to 15 keys.  $h = 2$ : 7 to 63 keys.  
2-3 tree height:  $\log_3 n \leq h \leq \log_2 n$ , so  $h \in \Theta(\log n)$ .

Search

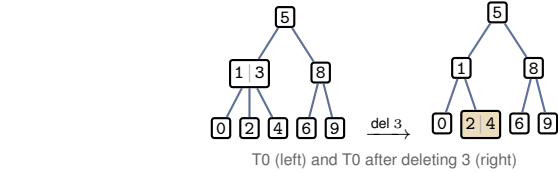
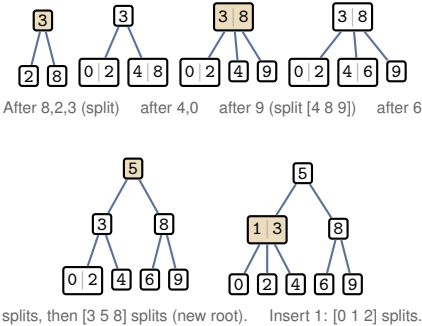
```
search(r, x):
    i <- 0
    while i < r.nitems and x > r.item[i]: i <- i + 1
    if i < r.nitems and x == r.item[i]: return (r, i)
    if r is a leaf: return null
    return search(r.children[i], x)
```

Insert

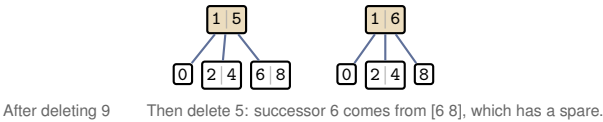
- 1. Walk down to the correct leaf and insert the key in sorted position.
- 2. If the node now has m keys (overflow), split: the median goes up to the parent; smaller keys form the left node, larger keys the right node.
- 3. If the parent overflows, split it too, and so on upward.
- 4. If the root splits, the median becomes a new root: height + 1 (the only way a B-tree gets taller).

For even m there are two middle candidates; follow the stated convention.

Trace: 2-3 tree, insert 8, 2, 3, 4, 0, 9, 6, 5, 1

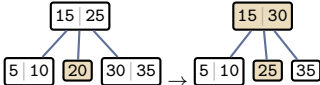


Then delete 9: [9] empties; left sibling [6] is minimal, so merge to [6 8]. Node [8] is empty; its sibling [1] is minimal, so merge [1] + 5. The root is empty and is removed: height drops.

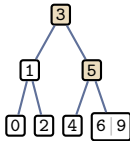


Rotation examples

Leaf rotation: delete 20 from [15 25][[5 10][20][30 35]]. The right sibling has a spare, so rotate left.



Internal rotation: delete 8 from T0. Successor 9; merge to [6 9]; node [9] is now empty. Left sibling [1 3] has a spare, so rotate right: 5 comes down, 3 goes up, and child [4] moves across.



How to: any delete trace

- (1) Find the key; if internal, swap in the successor.
- (2) Remove it from the leaf.
- (3) Compare the key count to the minimum.
- (4) Check the right sibling, then the left, then merge.
- (5) Re-check the parent; repeat upward.
- (6) Verify the invariants.

Invariants: internal nodes have 2 (root) or  $\lceil m/2 \rceil$  to m children; no null children;  $k + 1$  children  $\Leftrightarrow k$  sorted keys; all leaves on the same level.

10 Pseudocode Questions They Can Ask

Q1. Is this Heapify correct?

```
for i from 0 to floor(n/2) - 1: SiftDown(A, n, i) // top-down order
```

No. Max-heap on [1, 2, 3, 4]:  $i = 0$  gives [3, 2, 1, 4];  $i = 1$  gives [3, 4, 1, 2]. Root  $3 < 4$ . A node must be sifted after its subtrees are heaps, so go bottom-up.

Q2. Min-heap SiftDown swaps with the larger child. Still correct?

No. [9, 1, 2]: swapping 9 with 2 gives [2, 1, 9], and root  $2 > 1$ .

Q3. What does this return?

```
Mystery(r): if r == null: return 0
            return 1 + max(Mystery(r.left), Mystery(r.right))
```

The number of nodes on the longest path = height (in edges) + 1. For one node it returns 1, but  $h = 0$ .

Q4. What does this print, and in what time?

```
F(r): if r == null: return
      F(r.left); print(r.item); F(r.right)
```

In-order traversal: a BST's keys in sorted order,  $\Theta(n)$  (each node once).

Q5. Is this a correct BST check?

```
isBST(r): if r == null: return true
          if r.left != null and r.left.item > r.item: return false
          if r.right != null and r.right.item <= r.item: return false
          return isBST(r.left) and isBST(r.right)
```

No. It only checks children. Counterexample: 5 with left child 3, and 3 has right child 6 (6 is in 5's left subtree). Fix: pass bounds down.

```
isBST(r, lo, hi): if r == null: return true
                  if r.item < lo or r.item > hi: return false
                  return isBST(r.left, lo, r.item) and isBST(r.right, r.item, hi)
```

Q6. Write isHeap for a min-heap array.

```
isMinHeap(A, n):
    for i from 0 to floor(n/2) - 1:
        if 2i+1 < n and A[i] > A[2i+1]: return false
        if 2i+2 < n and A[i] > A[2i+2]: return false
    return true // Theta(n)
```

9 B-Tree Deletion

Algorithm

- Step 1: reach a leaf.** Key in a leaf: remove it. Key in an internal node: replace it with its in-order successor (min of the subtree to its right, always in a leaf) and delete that key from its leaf.
- Step 2:** if the leaf still has  $\geq$  the min number of keys, stop. (In a 2-3 tree, deleting from a 2-key leaf is free.)
- Step 3: fix underflow, in this order**
  - 1. **Rotate left** (right sibling has a spare key): separator moves down to the end of the deficient node; the right sibling's first key moves up.
  - 2. **Rotate right** (left sibling has a spare): separator moves down to the front; the left sibling's last key moves up.
  - 3. **Merge** (both adjacent siblings at the minimum): left node + separator + right node become one node. The parent loses a key.
    - Parent is the root and is now empty: delete it. Height - 1 (the only way it shrinks).
    - Parent is now deficient: repeat Step 3 on the parent.
- Siblings = adjacent children of the same parent only.
- **Internal-node rotation:** the child next to the key that moves up goes along to the deficient node.

Lecture examples, starting from T0

Delete 3: successor 4 replaces it; leaf [4] empties. No right sibling; left sibling [2] is at the minimum, so merge [2] + 4.